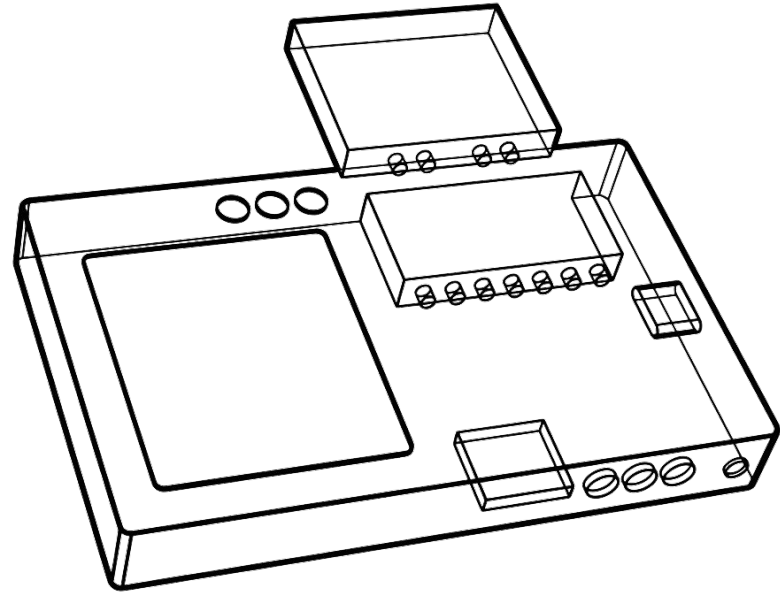


Diseño e implementación de un dispositivo IoT modular

Alejandro Rodríguez Mederos

Grado en Ingeniería Informática
ESIT · Universidad de La Laguna
La Laguna, junio 2026

Tutores: D. Iván Castilla · D. Santiago Torres



Estructura de la defensa

01 Motivación y objetivos

03 Arquitectura del sistema

05 Servidor HTTP embebido

07 Gestión energética

02 Estado del arte

04 DSL: lenguaje de dominio

06 Prototipo y demostración

08 Conclusiones (EN)

¿Por qué este dispositivo?

El problema

- Soluciones IoT altamente especializadas y cerradas
- El hardware no es reutilizable entre aplicaciones
- Adaptar el comportamiento exige recompilar el firmware
- Dependencia de servicios cloud para funcionar
- El usuario debe adaptarse al dispositivo, no al revés

La propuesta

- Plataforma base reutilizable para múltiples aplicaciones
- Ampliación rápida mediante módulos externos
- DSL: programar sin herramientas externas ni recompilación
- Servidor web embebido, configuración desde el ordenador
- El dispositivo se adapta al usuario

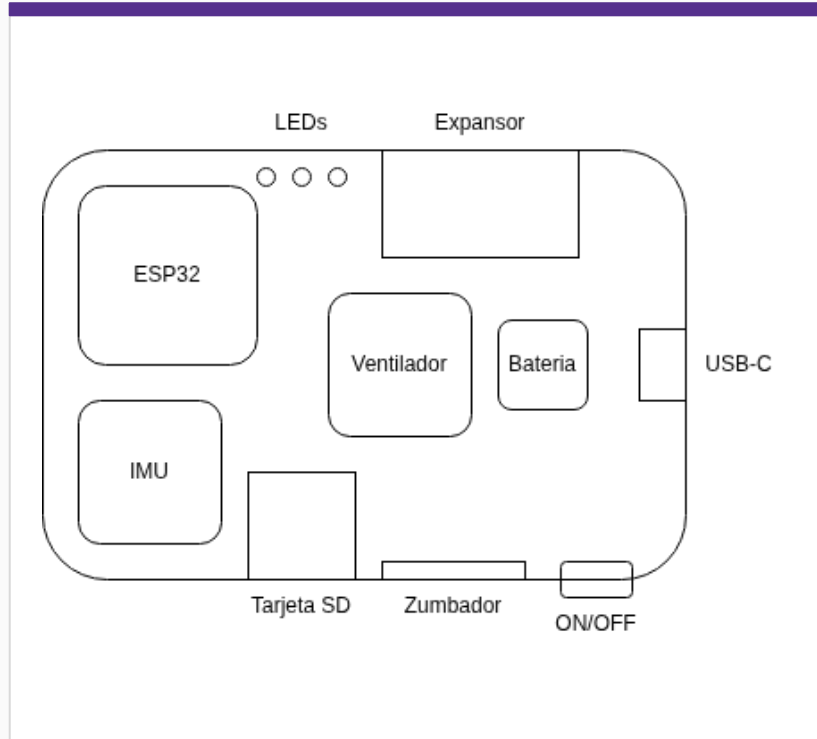
Comparativa de plataformas IoT

Plataforma	Dominio	Modular	Reconfig.	Cloud	Programación
IIoT Predictivo	Industrial	Media	Baja	Alta	–
Modular low-cost	Industrial	Alta	Baja	Alta	C/C++
Smart Agriculture	Agrícola	Baja	Baja	Alta	App externa
Est. Meteorológica	Ambiental	Media	Media	Media	Microservicios
Este TFG	Multi	Alta	Alta	Opcional	DSL embebido

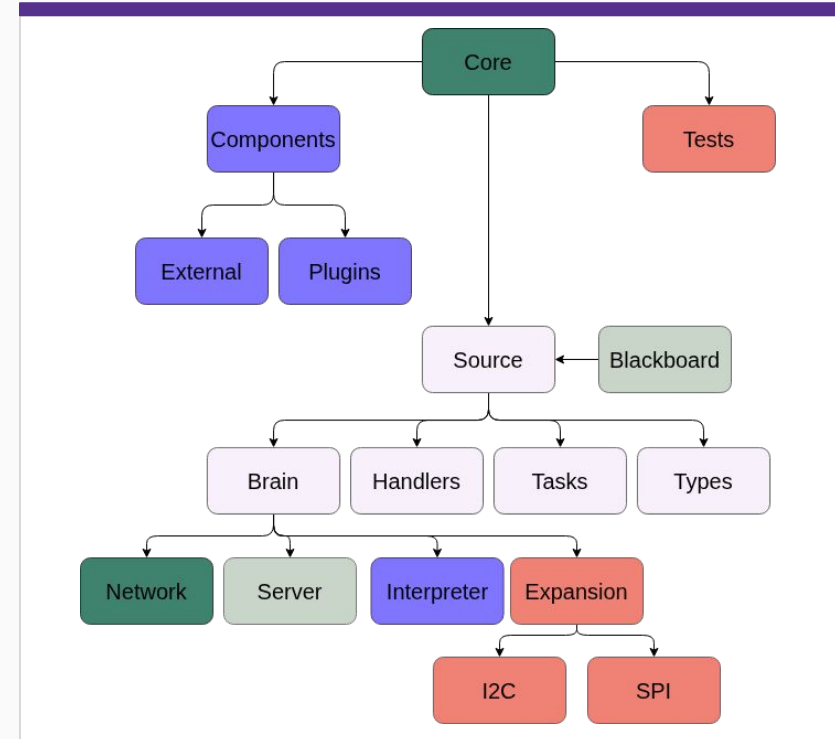
El dispositivo propuesto ocupa el cruce entre tres carencias comunes: especialización vertical, reconfigurabilidad limitada y dependencia de servicios externos. Ninguna solución analizada cubre los tres a la vez.

Arquitectura del sistema

Hardware



Software



Programar el hardware sin recompilar

Lexer + Interpreter

Tokenización y traducción a C++

Control de flujo

Variables, condicionales, bucles, operadores

Comunicación I2C

Lectura y escritura de registros.

Logs y archivos

Registros en microSD.

temperatura.str – ejemplo real del prototipo

```
FILE "temp.log"
```

```
I2C INIT
```

```
I2C WRITE 0x76 0xF4 0x27
```

```
WAIT 1
```

```
I2C READLE 0x76 0x88 2 -> dig_T1
```

```
I2C READLE 0x76 0x8A 2 -> dig_T2
```

```
I2C READLE 0x76 0x8C 2 -> dig_T3
```

```
I2C READ 0x76 0xFA 1 -> temp_msb
```

```
I2C READ 0x76 0xFB 1 -> temp_lsb
```

```
I2C READ 0x76 0xFC 1 -> temp_xlsb
```

```
temp_raw = (temp_msb << 12) | (temp_lsb << 4) | (temp_xlsb >> 4)
```

```
var1 = ((temp_raw / 8) - (dig_T1 * 2)) * dig_T2 / 2048
```

```
var2 = (((temp_raw / 16) - dig_T1) * ((temp_raw / 16) - dig_T1) / 4096) *  
dig_T3 / 16384 21
```

```
t_fine = var1 + var2
```

```
temp = (t_fine * 5 / 256) / 100
```

```
PRINT "Temperatura (grados C):" temp
```

Gestión desde el navegador

Modo Access Point (AP)

El dispositivo crea su propia red Wi-Fi.

Uso portátil y autónomo.

Acceso directo desde cualquier dispositivo.

Modo Station (STA)

Se conecta a una red existente.

Ideal para entornos fijos.

Acceso desde toda la red local.

Dashboard

Crear, editar, ejecutar y eliminar archivos

Logs

Visualización de registros generados

Documentación

Referencia del lenguaje integrada en el dispositivo

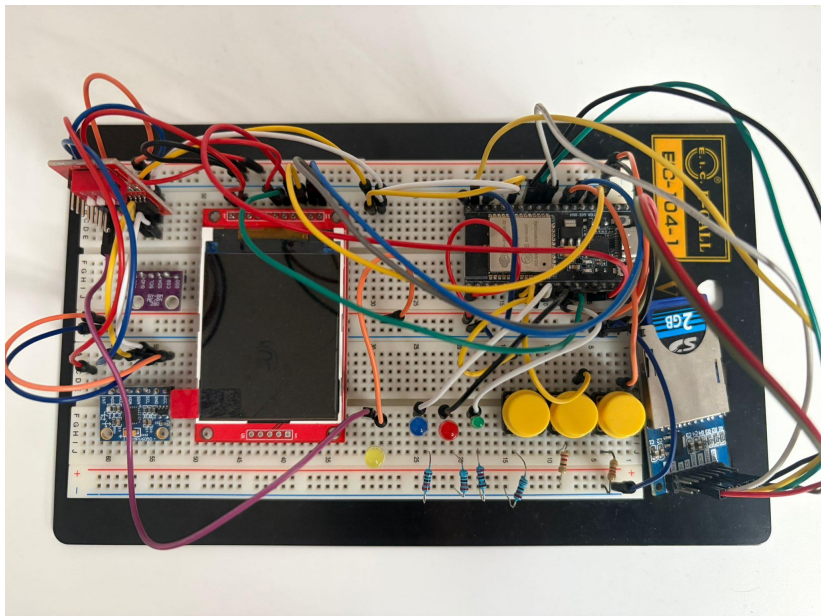
Configuración

Parámetros de red y modos de operación

Demostración del sistema en funcionamiento

VIDEO

Del diseño conceptual al hardware real



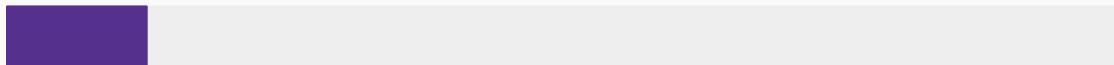
Funcionalidades validadas

- Ejecución de programas DSL en tiempo real
- Sensores temperatura y unidad de medida inercial vía I2C
- Dashboard web accesible desde navegador
- Logs con timestamp
- Navegación por menú
- Modos AP y STA configurables
- Pruebas automatizadas del núcleo

Autonomía estimada del sistema

Reposo

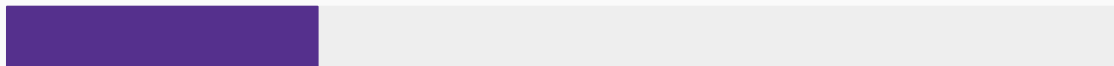
Pantalla atenuada,
ESP32 en bajo consumo



~45 mA
~190h

Modo STA

Conectado a
red Wi-Fi existente



~155 mA
~58h

Modo AP

Generando su
propia red Wi-Fi



~240 mA
~37h

Uso intensivo

Wi-Fi activo y
accesos frecuentes a SD



~385 mA
~23h

Valores calculados a partir de las especificaciones de los componentes. No medidos físicamente en el prototipo actual.

Conclusions & Future Work

Conclusions

- All initial objectives successfully achieved
- Modular, reusable IoT platform validated across multiple scenarios
- DSL proved effective, hardware interaction without recompilation
- Layered architecture ensures maintainability and scalability
- Energy analysis confirms viability for portable battery-powered use
- Prototype validates architecture, DSL and embedded server end-to-end

Future Work

- **DSL**
Event-driven execution
User-defined subroutines
SPI support + new data types
- **Hardware**
Custom PCB board
Full power & thermal validation
- **Software**
Using the tool ESP-Claw
- **System**
Implementation of a parser

Gracias

¿Preguntas?

ESP32 · DSL embebido · Servidor HTTP · Modular y portable